

1. Write a program using C/C++/Java to simulate the FCFS, SJF (pre-emptive as well as non pre-emptive approach). The scenario is: user may input n processes with respective CPU burst time and arrival time. System will ask the user to select the type of algorithm from the list mentioned above. System should display the waiting time for each process, average waiting time for the whole system, and final execution sequence.

```
//FCFS, SJF, SRTF

#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

#define INT_MAX 2147483647

struct Process {

    int id, arrivalTime, burstTime, priority;

    int completionTime, turnAroundTime, waitingTime, remainingTime;

};

void calculateTimes(vector<Process>& processes) {

    int total_wt = 0, total_tat = 0;

    for (auto& p : processes) {

        total_wt += p.waitingTime;

        total_tat += p.turnAroundTime;

    }

    cout << "Avg Waiting Time: " << (float)total_wt / processes.size() << endl;

    cout << "Avg Turn Around Time: " << (float)total_tat / processes.size() << endl;

}

void fcfs(vector<Process> processes) {

    int currentTime = 0;

    sort(processes.begin(), processes.end(), [](Process a, Process b) { return a.arrivalTime < b.arrivalTime; });

    for (auto& p : processes) {

        if (currentTime < p.arrivalTime) {

            currentTime = p.arrivalTime;

        }

    }

}
```

```

    currentTime += p.burstTime;

    p.completionTime = currentTime;

    p.turnAroundTime = p.completionTime - p.arrivalTime;

    p.waitingTime = p.turnAroundTime - p.burstTime;
}

cout << "FCFS Scheduling\n";
cout << "P.No\tArrival Time\tBurst Time\tCompletion Time\tTurn Around Time\tWaiting Time\n";
for (auto& p : processes) {
    cout << p.id << "\t\t" << p.arrivalTime << "\t\t" << p.burstTime << "\t\t" << p.completionTime
        << "\t\t" << p.turnAroundTime << "\t\t\t" << p.waitingTime << endl;
}
calculateTimes(processes);
}

void sjf(vector<Process> processes) {
    int n = processes.size(), currentTime = 0, completed = 0;
    vector<bool> isCompleted(n, false);

    while (completed != n) {
        int idx = -1, minBurst = INT_MAX;
        for (int i = 0; i < n; ++i) {
            if (!isCompleted[i] && processes[i].arrivalTime <= currentTime && processes[i].burstTime < minBurst) {
                minBurst = processes[i].burstTime;
                idx = i;
            }
        }
        if (idx != -1) {
            currentTime += processes[idx].burstTime;
            processes[idx].completionTime = currentTime;
            processes[idx].turnAroundTime = processes[idx].completionTime - processes[idx].arrivalTime;
            processes[idx].waitingTime = processes[idx].turnAroundTime - processes[idx].burstTime;
            isCompleted[idx] = true;
            completed++;
        } else {

```

```

        currentTime++;
    }
}

cout << "SJF Scheduling\n";
cout << "P.No\tArrival Time\tBurst Time\tCompletion Time\tTurn Around Time\tWaiting Time\n";
for (auto& p : processes) {
    cout << p.id << "\t\t" << p.arrivalTime << "\t\t" << p.burstTime << "\t\t" << p.completionTime
        << "\t\t" << p.turnAroundTime << "\t\t\t" << p.waitingTime << endl;
}
calculateTimes(processes);
}

void srtf(vector<Process> processes) {
    int n = processes.size(), currentTime = 0, completed = 0;
    for (auto& p : processes) p.remainingTime = p.burstTime;
    vector<bool> isCompleted(n, false);

    while (completed != n) {
        int idx = -1, minRemaining = INT_MAX;
        for (int i = 0; i < n; ++i) {
            if (!isCompleted[i] && processes[i].arrivalTime <= currentTime && processes[i].remainingTime <
minRemaining) {
                minRemaining = processes[i].remainingTime;
                idx = i;
            }
        }
        if (idx != -1) {
            processes[idx].remainingTime--;
            currentTime++;
            if (processes[idx].remainingTime == 0) {
                processes[idx].completionTime = currentTime;
                processes[idx].turnAroundTime = processes[idx].completionTime - processes[idx].arrivalTime;
                processes[idx].waitingTime = processes[idx].turnAroundTime - processes[idx].burstTime;
                isCompleted[idx] = true;
            }
        }
    }
}

```

```

        completed++;
    }
} else {
    currentTime++;
}
}

cout << "SRTF Scheduling\n";
cout << "P.No\tArrival Time\tBurst Time\tCompletion Time\tTurn Around Time\tWaiting Time\n";
for (auto& p : processes) {
    cout << p.id << "\t\t" << p.arrivalTime << "\t\t" << p.burstTime << "\t\t" << p.completionTime
        << "\t\t" << p.turnAroundTime << "\t\t\t" << p.waitingTime << endl;
}
calculateTimes(processes);
}

int main() {
    int n, choice;
    cout << "Enter number of processes: ";
    cin >> n;
    vector<Process> processes(n);
    for (int i = 0; i < n; i++) {
        processes[i].id = i + 1;
        cout << "Enter arrival time, burst time of process " << i + 1 << ": ";
        cin >> processes[i].arrivalTime >> processes[i].burstTime;
    }

    cout << "Select scheduling algorithm:\n1. FCFS\n2. SJF\n3. SRTF\n";
    cin >> choice;

    switch (choice) {
        case 1:
            fcfs(processes);
            break;
        case 2:

```

```

        sjf(processes);

        break;

    case 3:

        srtf(processes);

        break;

    default:

        cout << "Invalid choice\n";

}

return 0;

}

```

```

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab5$ ./Lab5
Enter number of processes: 4
Enter arrival time, burst time of process 1: 0 2
Enter arrival time, burst time of process 2: 1 2
Enter arrival time, burst time of process 3: 5 3
Enter arrival time, burst time of process 4: 6 4
Select scheduling algorithm:
1. FCFS
2. SJF
3. SRTF
1
FCFS Scheduling
P.No    Arrival Time    Burst Time    Completion Time    Turn Around Time    Waiting Time
1        0                2              2                  2                   0
2        1                2              4                  3                   1
3        5                3              8                  3                   0
4        6                4              12                 6                   2
Avg Waiting Time: 0.75
Avg Turn Around Time: 3.5

```

```

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab5$ ./Lab5
Enter number of processes: 4
Enter arrival time, burst time of process 1: 1 3
Enter arrival time, burst time of process 2: 2 4
Enter arrival time, burst time of process 3: 1 2
Enter arrival time, burst time of process 4: 4 4
Select scheduling algorithm:
1. FCFS
2. SJF
3. SRTF
2
SJF Scheduling
P.No    Arrival Time    Burst Time    Completion Time    Turn Around Time    Waiting Time
1        1                3              6                  5                   2
2        2                4              10                 8                   4
3        1                2              3                  2                   0
4        4                4              14                 10                  6
Avg Waiting Time: 3
Avg Turn Around Time: 6.25
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab5$

```

```

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab5$ ./Lab5
Enter number of processes: 4
Enter arrival time, burst time of process 1: 0 5
Enter arrival time, burst time of process 2: 1 3
Enter arrival time, burst time of process 3: 2 4
Enter arrival time, burst time of process 4: 4 1
Select scheduling algorithm:
1. FCFS
2. SJF
3. SRTF
3
SRTF Scheduling
P.No    Arrival Time    Burst Time    Completion Time    Turn Around Time    Waiting Time
1        0                5              9                  9                    4
2        1                3              4                  3                    0
3        2                4              13                 11                   7
4        4                1              5                  1                    0
Avg Waiting Time: 2.75
Avg Turn Around Time: 6
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab5$

```